Agents and Recursive Self-Improvement

A hands-on tutorial: from the tool-use loop to agents that improve themselves

Yingming Pu

LINs Lab, Westlake University

October 22, 2026

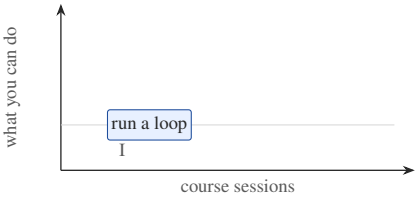


What this tutorial gives you



Three things you will be able to do afterwards.

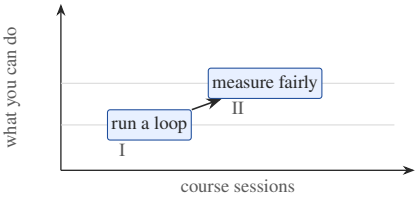
What this tutorial gives you



Three things you will be able to do afterwards.

1. Write a correct tool-use loop from scratch, with error and budget handling.

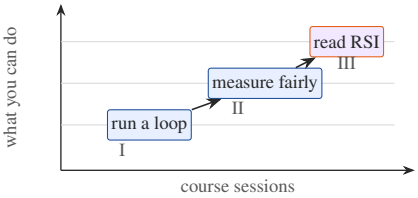
What this tutorial gives you



Three things you will be able to do afterwards.

1. Write a correct tool-use loop from scratch, with error and budget handling.
2. Wrap a course algorithm in an agent and evaluate it against a fair baseline.

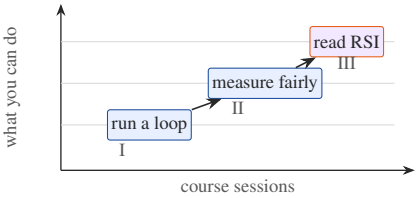
What this tutorial gives you



Three things you will be able to do afterwards.

1. Write a correct tool-use loop from scratch, with error and budget handling.
2. Wrap a course algorithm in an agent and evaluate it against a fair baseline.
3. Read an RSI paper and tell which claims the experiments support.

What this tutorial gives you

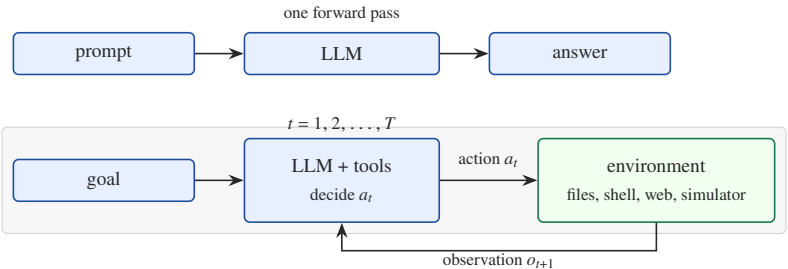


Three things you will be able to do afterwards.

1. Write a correct tool-use loop from scratch, with error and budget handling.
2. Wrap a course algorithm in an agent and evaluate it against a fair baseline.
3. Read an RSI paper and tell which claims the experiments support.

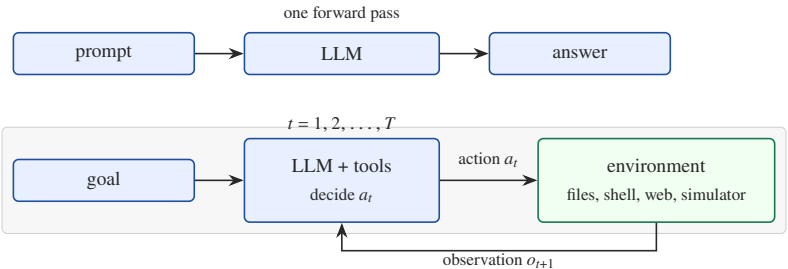
Assumed background. Probability, linear algebra, basic ML, comfortable Python. Code uses the Anthropic Python SDK; the loop shape is provider-independent.

Chatbot vs. agent: who chooses the next step?



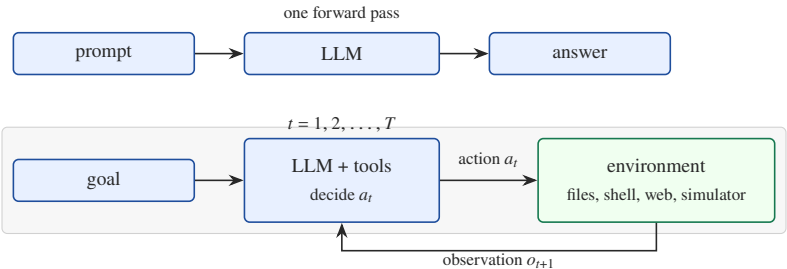
- A chatbot **answers**; an agent **pursues a goal by acting** until a stopping condition.

Chatbot vs. agent: who chooses the next step?



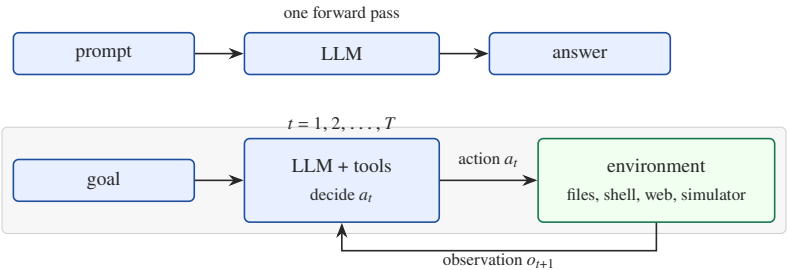
- A chatbot **answers**; an agent **pursues a goal by acting** until a stopping condition.
- Minimum ingredients: an LLM policy, tools, a loop, and a **termination test**.

Chatbot vs. agent: who chooses the next step?



- A chatbot **answers**; an agent **pursues a goal by acting** until a stopping condition.
- Minimum ingredients: an LLM policy, tools, a loop, and a **termination test**.
- Agency is not a property of the model alone: the same model is a chatbot or an agent depending on the harness.

Chatbot vs. agent: who chooses the next step?



- A chatbot **answers**; an agent **pursues a goal by acting** until a stopping condition.
- Minimum ingredients: an LLM policy, tools, a loop, and a **termination test**.
- Agency is not a property of the model alone: the same model is a chatbot or an agent depending on the harness.

Working definition. An agent alternates LLM calls with actions in an environment; the LLM chooses actions from a goal and the history of observations.

The formal view: an agent is a policy

A task is a partially observed Markov decision process (POMDP), and the LLM is the policy:

$$h_t = (g, o_1, a_1, \ldots, o_t), \qquad a_t \sim \pi_\theta(\cdot \mid h_t), \qquad o_{t+1} \sim \mathcal{E}(\cdot \mid h_t, a_t)$$

- g: goal prompt. h_t: the context window. $\mathcal{E}$: the environment (shell, API, simulator).

The formal view: an agent is a policy

A task is a partially observed Markov decision process (POMDP), and the LLM is the policy:

$$h_t = (g, o_1, a_1, \ldots, o_t), \qquad a_t \sim \pi_\theta(\cdot \mid h_t), \qquad o_{t+1} \sim \mathcal{E}(\cdot \mid h_t, a_t)$$

- g: goal prompt. h_t: the context window. $\mathcal{E}$: the environment (shell, API, simulator).
- Actions are structured: **tool calls**, **messages**, or **stop**. Tools are the action primitives.

The formal view: an agent is a policy

A task is a partially observed Markov decision process (POMDP), and the LLM is the policy:

$$h_t = (g, o_1, a_1, \ldots, o_t), \qquad a_t \sim \pi_\theta(\cdot \mid h_t), \qquad o_{t+1} \sim \mathcal{E}(\cdot \mid h_t, a_t)$$

- g: goal prompt. h_t: the context window. $\mathcal{E}$: the environment (shell, API, simulator).
- Actions are structured: **tool calls**, **messages**, or **stop**. Tools are the action primitives.
- The objective is a trajectory-level return, not a per-token loss:

$$\max_\theta \; \mathbb{E}_{\tau \sim \pi_\theta} \left[R(\tau) - \lambda \cdot \text{cost}(\tau) \right]$$

The formal view: an agent is a policy

A task is a partially observed Markov decision process (POMDP), and the LLM is the policy:

$$h_t = (g, o_1, a_1, \ldots, o_t), \qquad a_t \sim \pi_\theta(\cdot \mid h_t), \qquad o_{t+1} \sim \mathcal{E}(\cdot \mid h_t, a_t)$$

- g: goal prompt. h_t: the context window. $\mathcal{E}$: the environment (shell, API, simulator).
- Actions are structured: **tool calls**, **messages**, or **stop**. Tools are the action primitives.
- The objective is a trajectory-level return, not a per-token loss:

$$\max_\theta \; \mathbb{E}_{\tau \sim \pi_\theta} \left[R(\tau) - \lambda \cdot \text{cost}(\tau) \right]$$

Why the framing helps

It makes design choices explicit: what is observable, which actions exist, what reward is measured, what each step costs.

How it differs from supervised learning

No labels per step, sparse reward, long horizon, and the policy shifts the distribution it sees (non-stationarity).

One trace, two tool calls: what the model actually sees

user: improve val accuracy of the trainer

assistant: try a moderate lr first.

tool_use run_experiment(lr=0.1)

tool_result: val_acc = 0.81

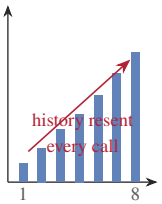
assistant: try lower lr, keep epochs.

tool_use run_experiment(lr=0.01)

tool_result: val_acc = 0.77

assistant: keep lr=0.1; report 0.81.

context size (schematic)



One trace, two tool calls: what the model actually sees

user: improve val accuracy of the trainer

assistant: try a moderate lr first.

tool_use run_experiment(lr=0.1)

tool_result: val_acc = 0.81

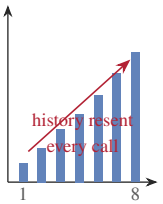
assistant: try lower lr, keep epochs.

tool_use run_experiment(lr=0.01)

tool_result: val_acc = 0.77

assistant: keep lr=0.1; report 0.81.

context size (schematic)



- Every turn appends to the same list; the model re-reads all of it on the next call.

One trace, two tool calls: what the model actually sees

user: improve val accuracy of the trainer

assistant: try a moderate lr first.

tool_use run_experiment(lr=0.1)

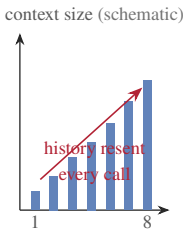
tool_result: val_acc = 0.81

assistant: try lower lr, keep epochs.

tool_use run_experiment(lr=0.01)

tool_result: val_acc = 0.77

assistant: keep lr=0.1; report 0.81.



- Every turn appends to the same list; the model re-reads all of it on the next call.
- The `tool_result` is the only information the model did not invent: it is where grounding comes from.

One trace, two tool calls: what the model actually sees

user: improve val accuracy of the trainer

assistant: try a moderate lr first.

tool_use run_experiment(lr=0.1)

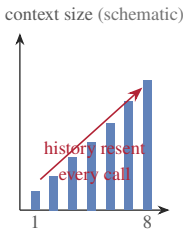
tool_result: val_acc = 0.81

assistant: try lower lr, keep epochs.

tool_use run_experiment(lr=0.01)

tool_result: val_acc = 0.77

assistant: keep lr=0.1; report 0.81.



- Every turn appends to the same list; the model re-reads all of it on the next call.
- The `tool_result` is the only information the model did not invent: it is where grounding comes from.
- Cost grows roughly quadratically in steps if you resend the full history each time.

Classic methods: five years in one picture

Track three columns that moved independently, instead of memorizing papers.

	2022–23	2023–24	2024–25	2025–26
Loop	reason + act	memory, search	pipelines	self-modification
Object	the answer	prompt, plan	workflow	own code
Evaluator	eyeball	output tests	benchmarks	exact verifier
Examples	ReAct, Toolformer	Reflexion, ToT	SWE-bench, DSPy	FunSearch, DGM

Classic methods: five years in one picture

Track three columns that moved independently, instead of memorizing papers.

	2022–23	2023–24	2024–25	2025–26
Loop	reason + act	memory, search	pipelines	self-modification
Object	the answer	prompt, plan	workflow	own code
Evaluator	eyeball	output tests	benchmarks	exact verifier
Examples	ReAct, Toolformer	Reflexion, ToT	SWE-bench, DSPy	FunSearch, DGM

- The **control loop** has not changed since 2022: thought, tool call, observation, repeat.

Classic methods: five years in one picture

Track three columns that moved independently, instead of memorizing papers.

	2022–23	2023–24	2024–25	2025–26
Loop	reason + act	memory, search	pipelines	self-modification
Object	the answer	prompt, plan	workflow	own code
Evaluator	eyeball	output tests	benchmarks	exact verifier
Examples	ReAct, Toolformer	Reflexion, ToT	SWE-bench, DSPy	FunSearch, DGM

- The **control loop** has not changed since 2022: thought, tool call, observation, repeat.
- What changed is the **object of improvement**, and with it the **evaluator** that makes improvement measurable.

Classic methods: five years in one picture

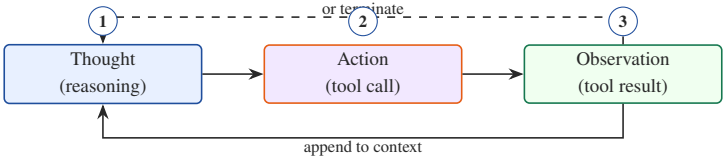
Track three columns that moved independently, instead of memorizing papers.

	2022–23	2023–24	2024–25	2025–26
Loop	reason + act	memory, search	pipelines	self-modification
Object	the answer	prompt, plan	workflow	own code
Evaluator	eyeball	output tests	benchmarks	exact verifier
Examples	ReAct, Toolformer	Reflexion, ToT	SWE-bench, DSPy	FunSearch, DGM

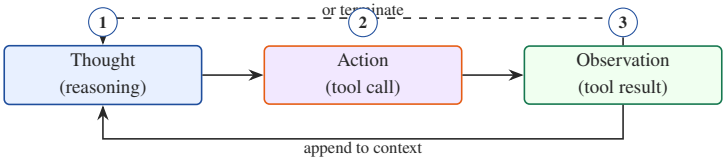
- The **control loop** has not changed since 2022: thought, tool call, observation, repeat.
- What changed is the **object of improvement**, and with it the **evaluator** that makes improvement measurable.

Reading strategy. Locate any new agent paper in this table: which loop, which object, which verifier. Most "new paradigms" are a new object with an old loop.

The loop: thought, action, observation

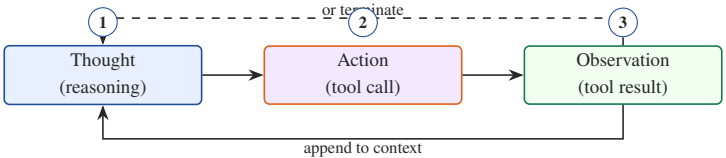


The loop: thought, action, observation



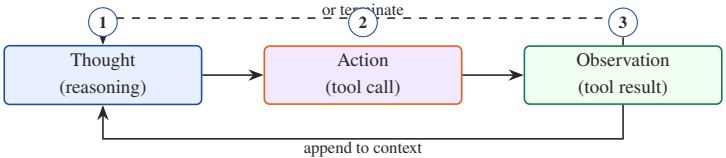
- **ReAct** interleaves free-form reasoning with actions, so the model can revise its plan after each observation arXiv:2210.03629.

The loop: thought, action, observation



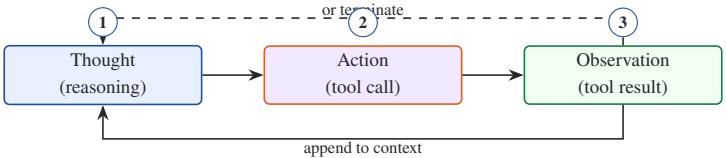
- **ReAct** interleaves free-form reasoning with actions, so the model can revise its plan after each observation arXiv:2210.03629.
- Reasoning without acting hallucinates state; acting without reasoning repeats mistakes.

The loop: thought, action, observation



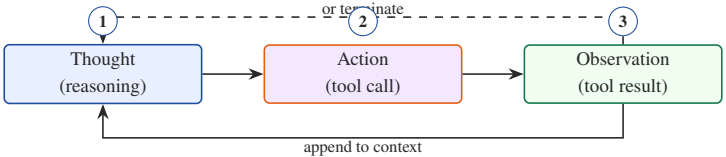
- **ReAct** interleaves free-form reasoning with actions, so the model can revise its plan after each observation arXiv:2210.03629.
- Reasoning without acting hallucinates state; acting without reasoning repeats mistakes.
- Thinking may be hidden (extended thinking) but the control flow is unchanged.

The loop: thought, action, observation



- **ReAct** interleaves free-form reasoning with actions, so the model can revise its plan after each observation arXiv:2210.03629.
- Reasoning without acting hallucinates state; acting without reasoning repeats mistakes.
- Thinking may be hidden (extended thinking) but the control flow is unchanged.
- Each iteration is one model call plus one environment step: cost grows with T.

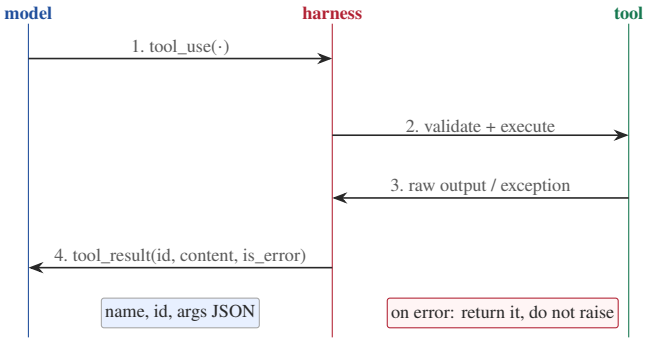
The loop: thought, action, observation



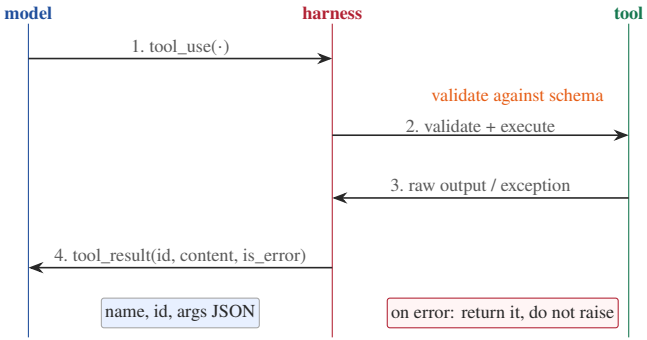
- **ReAct** interleaves free-form reasoning with actions, so the model can revise its plan after each observation arXiv:2210.03629.
- Reasoning without acting hallucinates state; acting without reasoning repeats mistakes.
- Thinking may be hidden (extended thinking) but the control flow is unchanged.
- Each iteration is one model call plus one environment step: cost grows with T.

Rule of thumb. If the task can be solved by one prompt, do not build an agent. Agents pay off when the task needs *grounded* intermediate steps.

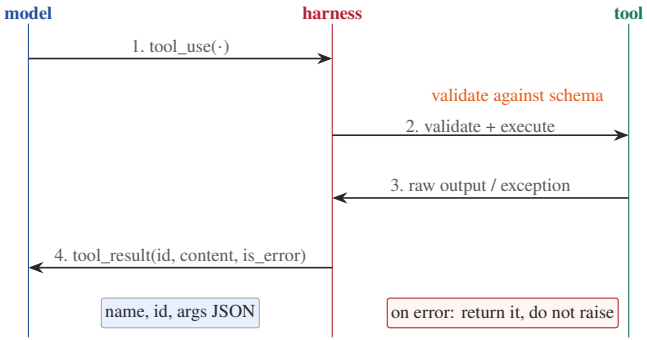
Anatomy of one tool call



Anatomy of one tool call

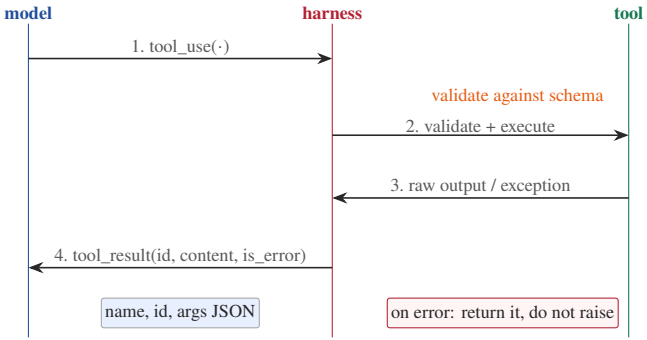


Anatomy of one tool call



the id is the only link between a call and its result

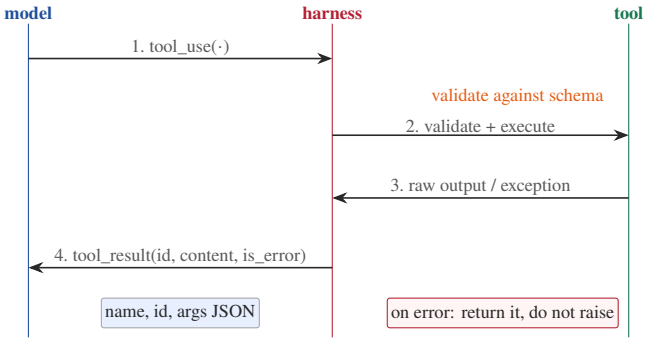
Anatomy of one tool call



the id is the only link between a call and its result

- The model never executes code; it emits a request. The harness is the trusted boundary.

Anatomy of one tool call



the id is the only link between a call and its result

- The model never executes code; it emits a request. The harness is the trusted boundary.
- Errors are observations: a tool that raises destroys the transcript; one that returns an error teaches the model.

Tools: the API contract that makes acting possible

A tool is three things: a **name**, a **natural-language description** the model reads, and a **JSON schema** the runtime validates.

```
{
  "name": "run_training",
  "description": "Train one config on the
     fixed train split; return metrics.",
  "input_schema": {
    "type": "object",
    "properties": {
      "model": {"type": "string"},
      "lr": {"type": "number"},
      "epochs": {"type": "integer"}
    },
    "required": ["model", "lr", "epochs"]
  }
}
```

Related: Toolformer learns when to call APIs arXiv:2302.04761; MCP standardizes how tools are served to many agents.

Tools: the API contract that makes acting possible

A tool is three things: a **name**, a **natural-language description** the model reads, and a **JSON schema** the runtime validates.

- The model never executes anything. It emits a call; *your* code executes it and returns the result as an observation.

```
{
  "name": "run_training",
  "description": "Train one config on the
     fixed train split; return metrics.",
  "input_schema": {
    "type": "object",
    "properties": {
      "model": {"type": "string"},
      "lr": {"type": "number"},
      "epochs": {"type": "integer"}
    },
    "required": ["model", "lr", "epochs"]
  }
}
```

Related: Toolformer learns when to call APIs arXiv:2302.04761; MCP standardizes how tools are served to many agents.

Tools: the API contract that makes acting possible

A tool is three things: a **name**, a **natural-language description** the model reads, and a **JSON schema** the runtime validates.

- The model never executes anything. It emits a call; *your* code executes it and returns the result as an observation.
- Errors are observations too: return them with an error flag instead of crashing the loop.

```
{
  "name": "run_training",
  "description": "Train one config on the
     fixed train split; return metrics.",
  "input_schema": {
    "type": "object",
    "properties": {
      "model": {"type": "string"},
      "lr": {"type": "number"},
      "epochs": {"type": "integer"}
    },
    "required": ["model", "lr", "epochs"]
  }
}
```

Related: Toolformer learns when to call APIs arXiv:2302.04761; MCP standardizes how tools are served to many agents.

Tools: the API contract that makes acting possible

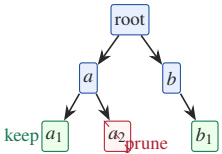
A tool is three things: a **name**, a **natural-language description** the model reads, and a **JSON schema** the runtime validates.

- The model never executes anything. It emits a call; *your* code executes it and returns the result as an observation.
- Errors are observations too: return them with an error flag instead of crashing the loop.
- Few, orthogonal, typed tools beat a large messy set. A shell is powerful but opaque; promote an action to a dedicated tool when you need to gate, audit or parallelize it.

```
{
  "name": "run_training",
  "description": "Train one config on the
     fixed train split; return metrics.",
  "input_schema": {
    "type": "object",
    "properties": {
      "model": {"type": "string"},
      "lr": {"type": "number"},
      "epochs": {"type": "integer"}
    },
    "required": ["model", "lr", "epochs"]
  }
}
```

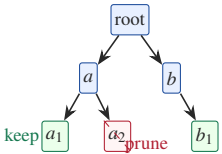
Related: Toolformer learns when to call APIs arXiv:2302.04761; MCP standardizes how tools are served to many agents.

Planning: spending compute at inference time



score each node, expand the best (≈ ToT)

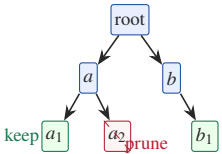
Planning: spending compute at inference time



score each node, expand the best (≈ ToT)

- **Plan-and-execute**: write a plan, run it, replan on failure. Few calls, but a bad plan is expensive.

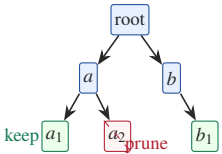
Planning: spending compute at inference time



score each node, expand the best (≈ ToT)

- **Plan-and-execute**: write a plan, run it, replan on failure. Few calls, but a bad plan is expensive.
- **Tree of Thoughts**: expand and score partial nodes arXiv:2305.10601. Pays off only when scoring is cheap.

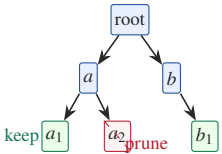
Planning: spending compute at inference time



score each node, expand the best (≈ ToT)

- **Plan-and-execute**: write a plan, run it, replan on failure. Few calls, but a bad plan is expensive.
- **Tree of Thoughts**: expand and score partial nodes arXiv:2305.10601. Pays off only when scoring is cheap.
- **Search over programs**: replace self-judgment with an external evaluator (next section).

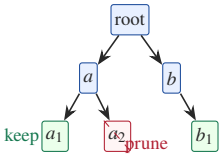
Planning: spending compute at inference time



score each node, expand the best (≈ ToT)

- **Plan-and-execute**: write a plan, run it, replan on failure. Few calls, but a bad plan is expensive.
- **Tree of Thoughts**: expand and score partial nodes arXiv:2305.10601. Pays off only when scoring is cheap.
- **Search over programs**: replace self-judgment with an external evaluator (next section).
- Planning is a compute decision: more branching means more calls. Always report the budget.

Planning: spending compute at inference time

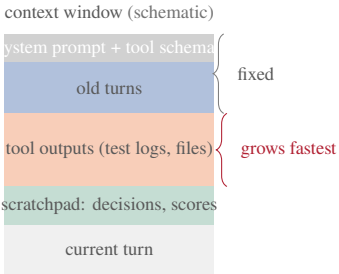


score each node, expand the best (≈ ToT)

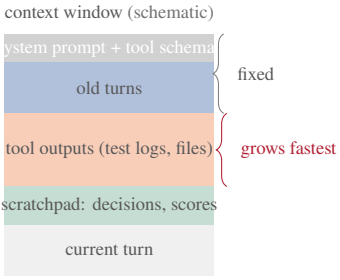
- **Plan-and-execute**: write a plan, run it, replan on failure. Few calls, but a bad plan is expensive.
- **Tree of Thoughts**: expand and score partial nodes arXiv:2305.10601. Pays off only when scoring is cheap.
- **Search over programs**: replace self-judgment with an external evaluator (next section).
- Planning is a compute decision: more branching means more calls. Always report the budget.

Diagnostic. If extra test-time compute does not improve the score, the bottleneck is the evaluator or the action space, not the planner.

Memory: context is a budget, not a database

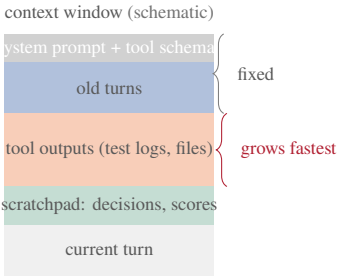


Memory: context is a budget, not a database



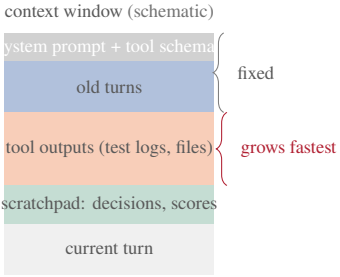
- **Short-term**: the transcript. Tool outputs dominate the token count; a test log can be 100× the request.

Memory: context is a budget, not a database



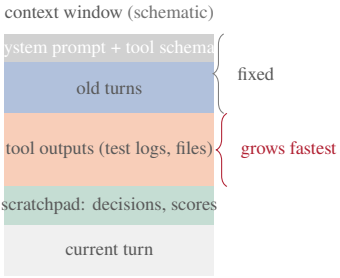
- **Short-term**: the transcript. Tool outputs dominate the token count; a test log can be 100× the request.
- Manage it explicitly: truncate noisy outputs, summarize old turns, keep a structured scratchpad of decisions.

Memory: context is a budget, not a database



- **Short-term**: the transcript. Tool outputs dominate the token count; a test log can be 100× the request.
- Manage it explicitly: truncate noisy outputs, summarize old turns, keep a structured scratchpad of decisions.
- **Long-term**: files, databases, a skill library (Voyager arXiv:2305.16291). Typed records beat free-text summaries: auditable and reloadable.

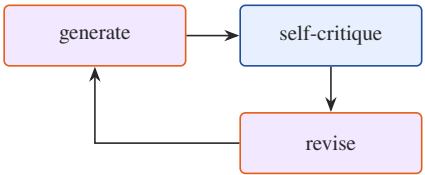
Memory: context is a budget, not a database



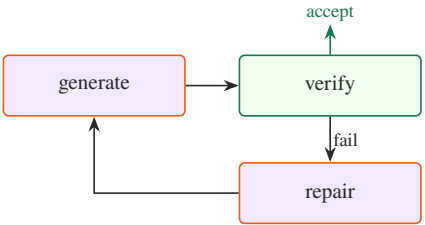
- **Short-term**: the transcript. Tool outputs dominate the token count; a test log can be 100× the request.
- Manage it explicitly: truncate noisy outputs, summarize old turns, keep a structured scratchpad of decisions.
- **Long-term**: files, databases, a skill library (Voyager arXiv:2305.16291). Typed records beat free-text summaries: auditable and reloadable.
- Context is a cache, not a database: every token costs money and dilutes attention.

Reflection: useful only with an external verifier

Two loops that look similar but behave very differently:



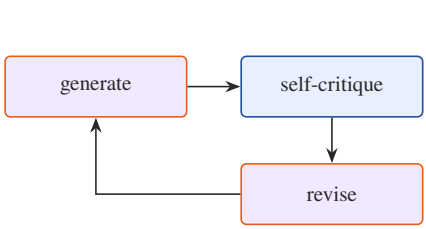
same blind spots: often no gain



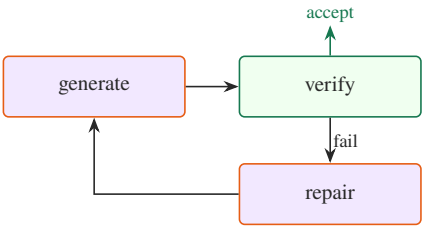
errors are found by code, not opinion

Reflection: useful only with an external verifier

Two loops that look similar but behave very differently:



same blind spots: often no gain

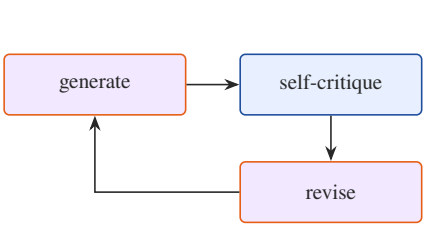


errors are found by code, not opinion

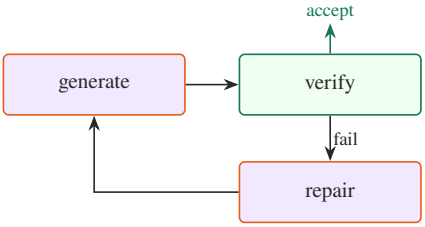
- **Self-Refine**: the same model critiques and revises arXiv:2303.17651. Cheap; works for explicit criteria (style, format).

Reflection: useful only with an external verifier

Two loops that look similar but behave very differently:



same blind spots: often no gain

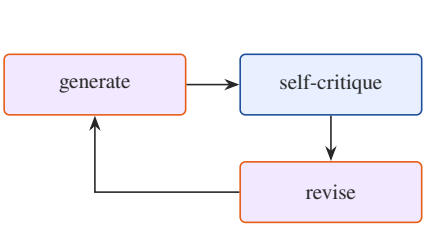


errors are found by code, not opinion

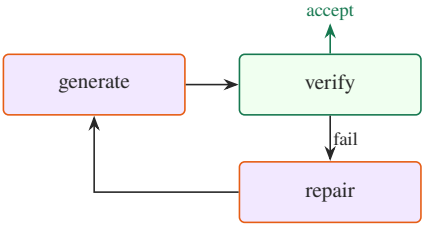
- **Self-Refine**: the same model critiques and revises arXiv:2303.17651. Cheap; works for explicit criteria (style, format).
- **Reflexion**: after failure, the agent writes a verbal lesson and retries arXiv:2303.11366. Works because failure is measured.

Reflection: useful only with an external verifier

Two loops that look similar but behave very differently:



same blind spots: often no gain

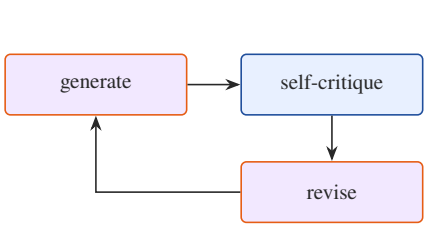


errors are found by code, not opinion

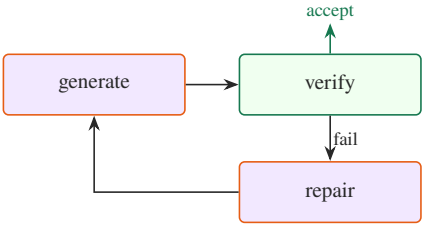
- **Self-Refine**: the same model critiques and revises arXiv:2303.17651. Cheap; works for explicit criteria (style, format).
- **Reflexion**: after failure, the agent writes a verbal lesson and retries arXiv:2303.11366. Works because failure is measured.
- Without external feedback, intrinsic self-correction often makes answers worse arXiv:2310.01798.

Reflection: useful only with an external verifier

Two loops that look similar but behave very differently:



same blind spots: often no gain

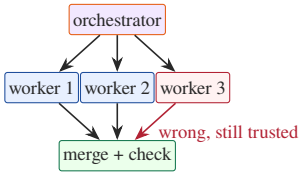


errors are found by code, not opinion

- **Self-Refine**: the same model critiques and revises arXiv:2303.17651. Cheap; works for explicit criteria (style, format).
- **Reflexion**: after failure, the agent writes a verbal lesson and retries arXiv:2303.11366. Works because failure is measured.
- Without external feedback, intrinsic self-correction often makes answers worse arXiv:2310.01798.

Design rule. "Generate, verify, repair" beats "generate, reflect". The verifier must be independent of the generator.

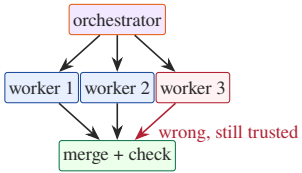
Multi-agent systems: parallelism, not magic



fan-out is easy; verification of fan-in is not (schematic)

Frameworks: AutoGen arXiv:2308.08155, MetaGPT arXiv:2308.00352.

Multi-agent systems: parallelism, not magic

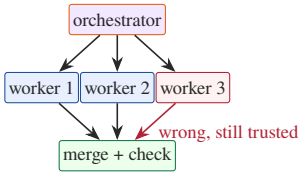


fan-out is easy; verification of fan-in is not (schematic)

- **Orchestrator–workers**: split independent subtasks, then merge. Real gain: context isolation.

Frameworks: AutoGen arXiv:2308.08155, MetaGPT arXiv:2308.00352.

Multi-agent systems: parallelism, not magic

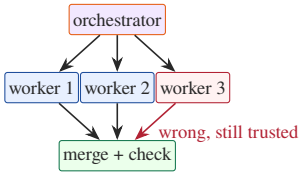


fan-out is easy; verification of fan-in is not (schematic)

- **Orchestrator–workers**: split independent subtasks, then merge. Real gain: context isolation.
- **Generator–critic**: separate contexts for producing and checking; reduces self-confirmation.

Frameworks: AutoGen arXiv:2308.08155, MetaGPT arXiv:2308.00352.

Multi-agent systems: parallelism, not magic

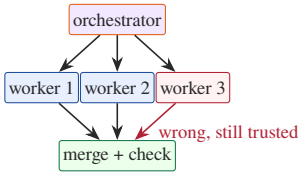


fan-out is easy; verification of fan-in is not (schematic)

- **Orchestrator–workers**: split independent subtasks, then merge. Real gain: context isolation.
- **Generator–critic**: separate contexts for producing and checking; reduces self-confirmation.
- Failure modes: error propagation, coordination overhead, role drift (MAST arXiv:2503.13657).

Frameworks: AutoGen arXiv:2308.08155, MetaGPT arXiv:2308.00352.

Multi-agent systems: parallelism, not magic



fan-out is easy; verification of fan-in is not (schematic)

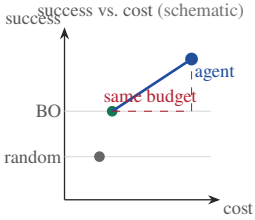
- **Orchestrator–workers**: split independent subtasks, then merge. Real gain: context isolation.
- **Generator–critic**: separate contexts for producing and checking; reduces self-confirmation.
- Failure modes: error propagation, coordination overhead, role drift (MAST arXiv:2503.13657).
- Start with one agent and one loop. Add agents only after measuring that a single context is the bottleneck.

Frameworks: AutoGen arXiv:2308.08155, MetaGPT arXiv:2308.00352.

Evaluating an agent: success, cost, reliability

An agent result is a *distribution over trajectories*. Report three axes:

Axis	Measure	Metric
Outcome	task solved?	pass@1, tests
Cost	tokens, time	$ per task
Reliability	run k times	pass^k



Read the plot

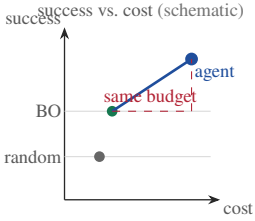
A gain counts only if it survives the dashed budget line.

Evaluating an agent: success, cost, reliability

An agent result is a *distribution over trajectories*. Report three axes:

Axis	Measure	Metric
Outcome	task solved?	pass@1, tests
Cost	tokens, time	$ per task
Reliability	run k times	pass^k

- Benchmarks: SWE-bench arXiv:2310.06770, WebArena arXiv:2307.13854, τ-bench arXiv:2406.12045.



Read the plot

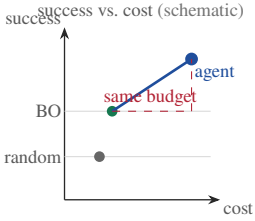
A gain counts only if it survives the dashed budget line.

Evaluating an agent: success, cost, reliability

An agent result is a *distribution over trajectories*. Report three axes:

Axis	Measure	Metric
Outcome	task solved?	pass@1, tests
Cost	tokens, time	$ per task
Reliability	run k times	pass^k

- Benchmarks: SWE-bench arXiv:2310.06770, WebArena arXiv:2307.13854, τ-bench arXiv:2406.12045.
- Report variance over seeds and failures, not the best run.



Read the plot

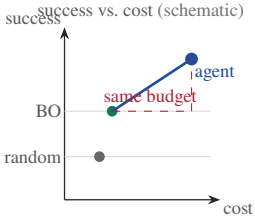
A gain counts only if it survives the dashed budget line.

Evaluating an agent: success, cost, reliability

An agent result is a *distribution over trajectories*. Report three axes:

Axis	Measure	Metric
Outcome	task solved?	pass@1, tests
Cost	tokens, time	$ per task
Reliability	run k times	pass^k

- Benchmarks: SWE-bench arXiv:2310.06770, WebArena arXiv:2307.13854, τ-bench arXiv:2406.12045.
- Report variance over seeds and failures, not the best run.
- Check contamination: hold out tasks before any gain claim.



Read the plot

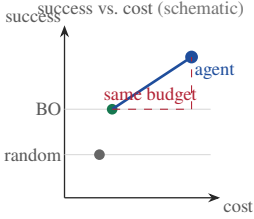
A gain counts only if it survives the dashed budget line.

Evaluating an agent: success, cost, reliability

An agent result is a *distribution over trajectories*. Report three axes:

Axis	Measure	Metric
Outcome	task solved?	pass@1, tests
Cost	tokens, time	$ per task
Reliability	run k times	pass^k

- Benchmarks: SWE-bench arXiv:2310.06770, WebArena arXiv:2307.13854, τ-bench arXiv:2406.12045.
- Report variance over seeds and failures, not the best run.
- Check contamination: hold out tasks before any gain claim.

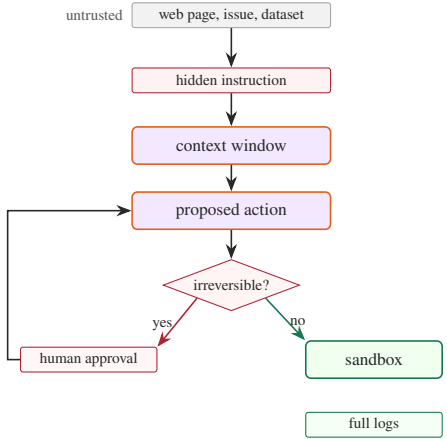


Read the plot

A gain counts only if it survives the dashed budget line.

Minimum credible protocol. Fixed task set, fixed budget, $\geq$ 3 seeds, cost axis, and a baseline that uses the same model.

Safety and operations: the agent has your permissions



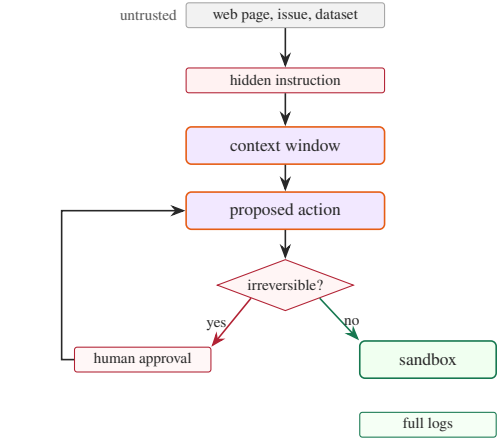
Threats

- Prompt injection via retrieved text.
- Irreversible actions: delete, send, spend.
- Secret leakage; runaway loops.

Controls

- Least privilege: read-only default.
- Sandbox the executor; no network.
- Budget and step caps; full logs.

Safety and operations: the agent has your permissions



Threats

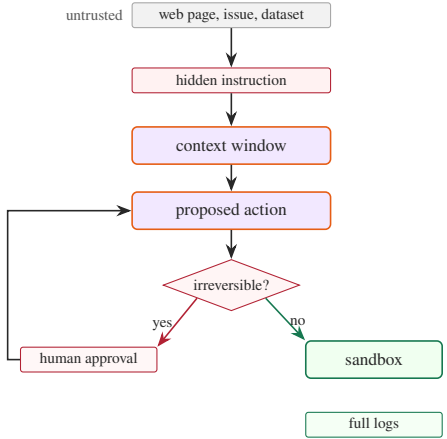
- Prompt injection via retrieved text.
- Irreversible actions: delete, send, spend.
- Secret leakage; runaway loops.

Controls

- Least privilege: read-only default.
- Sandbox the executor; no network.
- Budget and step caps; full logs.

- The model proposes; the harness enforces policy.

Safety and operations: the agent has your permissions



Threats

- Prompt injection via retrieved text.
- Irreversible actions: delete, send, spend.
- Secret leakage; runaway loops.

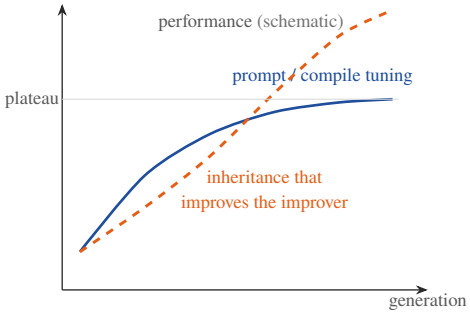
Controls

- Least privilege: read-only default.
- Sandbox the executor; no network.
- Budget and step caps; full logs.

- The model proposes; the harness enforces policy.
- **Reversible actions run automatically; irreversible ones need a human gate.**

What “recursive self-improvement” actually means

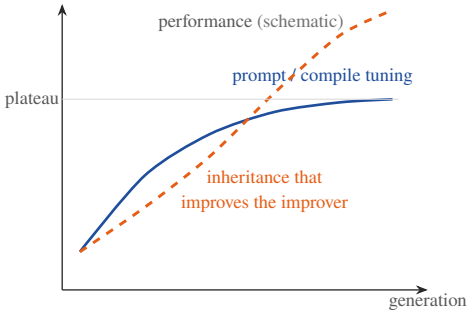
Definition. RSI is a loop in which the improvement is *used to improve the improver*: generation $t+1$ is better at creating improvements than generation t, not merely better at the task.



What “recursive self-improvement” actually means

Definition. RSI is a loop in which the improvement is *used to improve the improver*: generation $t+1$ is better at creating improvements than generation t, not merely better at the task.

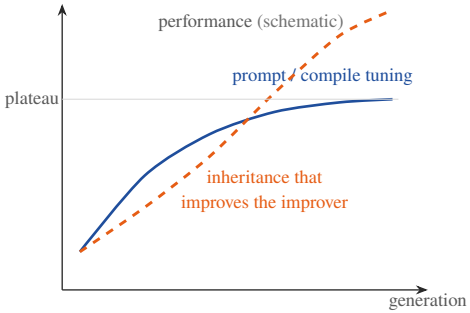
- Every loop below is an *improvement loop*. Only the last one *compounds*.



What “recursive self-improvement” actually means

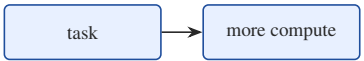
Definition. RSI is a loop in which the improvement is *used to improve the improver*: generation $t+1$ is better at creating improvements than generation t, not merely better at the task.

- Every loop below is an *improvement loop*. Only the last one *compounds*.
- The key question for any RSI claim: **what state is inherited, and is the next generation faster?**



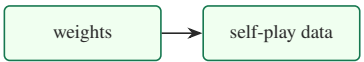
Three loops that are often confused

Inference-time



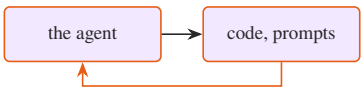
nothing persists

Training-time



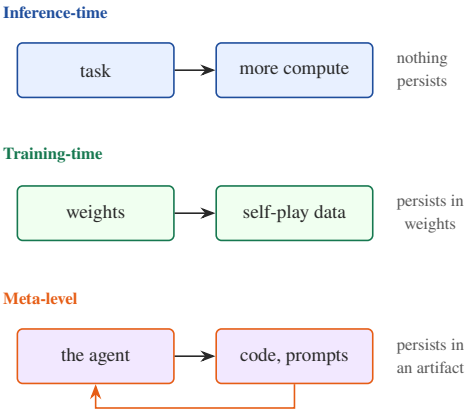
persists in weights

Meta-level



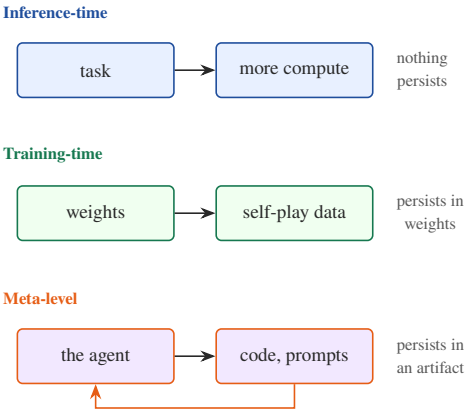
persists in an artifact

Three loops that are often confused



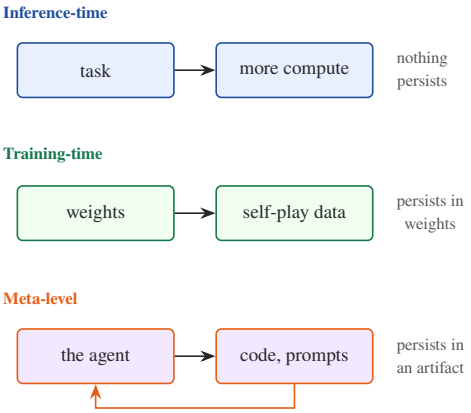
- Inference-time scaling improves one answer; it leaves no trace the next task can use.

Three loops that are often confused



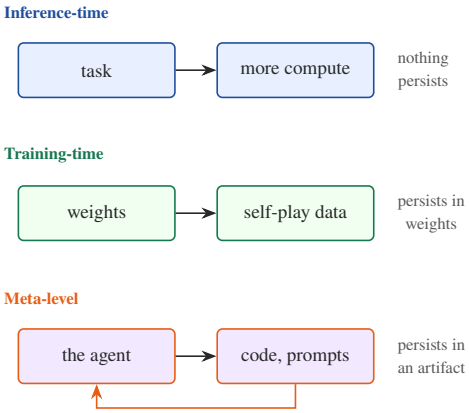
- Inference-time scaling improves one answer; it leaves no trace the next task can use.
- Training-time improvement persists, but each generation is a retraining event.

Three loops that are often confused



- Inference-time scaling improves one answer; it leaves no trace the next task can use.
- Training-time improvement persists, but each generation is a retraining event.
- Meta-level improvement is *cheap to inherit*: copy the artifact, run again.

Three loops that are often confused

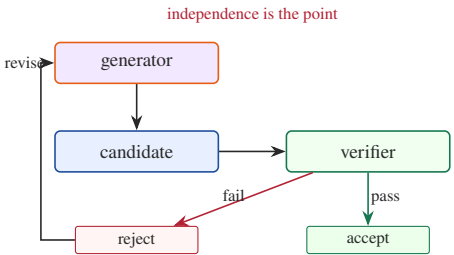


- Inference-time scaling improves one answer; it leaves no trace the next task can use.
- Training-time improvement persists, but each generation is a retraining event.
- Meta-level improvement is *cheap to inherit*: copy the artifact, run again.

Test. What survives the episode? Nothing: inference. Weights: training. A file the next run reads: meta-level.

Why the verifier, not the model, is the bottleneck

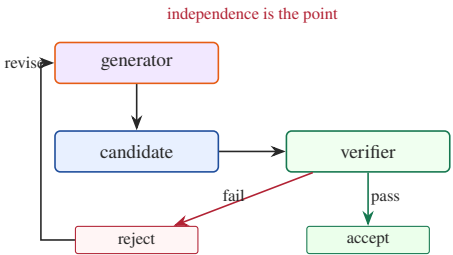
Signal	What actually happens
None	random walk; “improvements” are noise
Model-as-judge	self-confirmation: same blind spots
Exact verifier	real gain; ceiling is its coverage



Why the verifier, not the model, is the bottleneck

Signal	What actually happens
None	random walk; “improvements” are noise
Model-as-judge	self-confirmation: same blind spots
Exact verifier	real gain; ceiling is its coverage

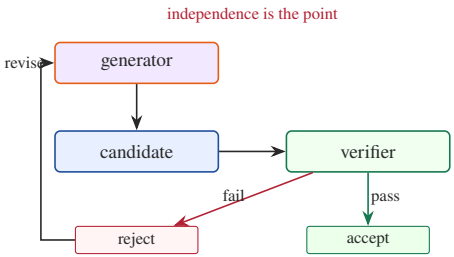
- A verifier buys a search signal, but it also **defines the goal**: agents optimize exactly what is measured.



Why the verifier, not the model, is the bottleneck

Signal	What actually happens
None	random walk; “improvements” are noise
Model-as-judge	self-confirmation: same blind spots
Exact verifier	real gain; ceiling is its coverage

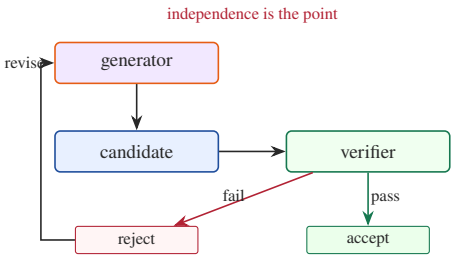
- A verifier buys a search signal, but it also **defines the goal**: agents optimize exactly what is measured.
- Goodhart’s law as an engineering constraint: a competent agent finds the loophole first.



Why the verifier, not the model, is the bottleneck

Signal	What actually happens
None	random walk; “improvements” are noise
Model-as-judge	self-confirmation: same blind spots
Exact verifier	real gain; ceiling is its coverage

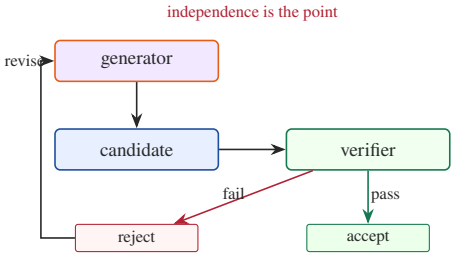
- A verifier buys a search signal, but it also **defines the goal**: agents optimize exactly what is measured.
- Goodhart’s law as an engineering constraint: a competent agent finds the loophole first.
- Practical implication: **spend more effort on the checker than on the prompt**.



Why the verifier, not the model, is the bottleneck

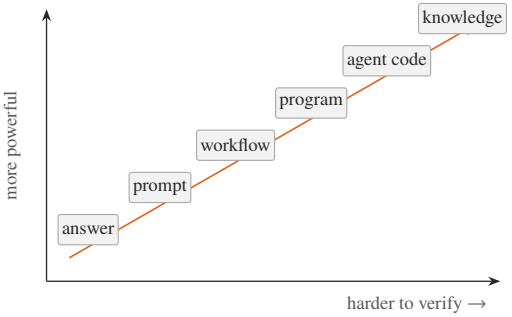
Signal	What actually happens
None	random walk; “improvements” are noise
Model-as-judge	self-confirmation: same blind spots
Exact verifier	real gain; ceiling is its coverage

- A verifier buys a search signal, but it also **defines the goal**: agents optimize exactly what is measured.
- Goodhart’s law as an engineering constraint: a competent agent finds the loophole first.
- Practical implication: **spend more effort on the checker than on the prompt**.



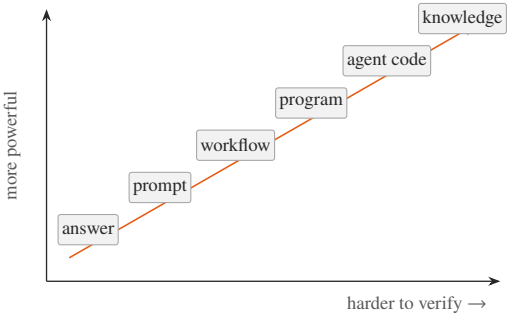
Rule. Before adding a mechanism, ask what signal it optimizes and who could corrupt that signal. If the answer includes the agent, the mechanism is unsafe.

The improvement object: where the change happens



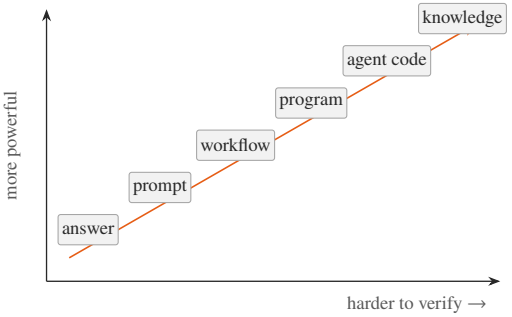
- Every method picks an **object** to improve. The object determines the ceiling.

The improvement object: where the change happens



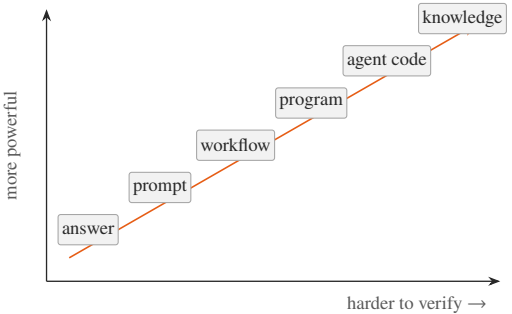
- Every method picks an **object** to improve. The object determines the ceiling.
- Early objects are easy to score but limited: an answer cannot transfer to the next task.

The improvement object: where the change happens



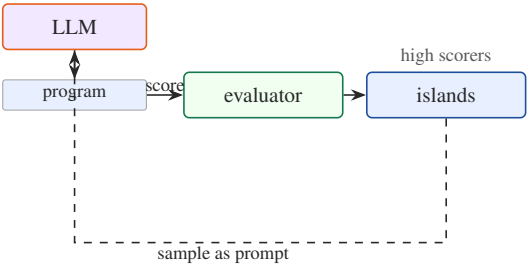
- Every method picks an **object** to improve. The object determines the ceiling.
- Early objects are easy to score but limited: an answer cannot transfer to the next task.
- Later objects are powerful but hard to verify: an agent's "better work process" has no unit test.

The improvement object: where the change happens

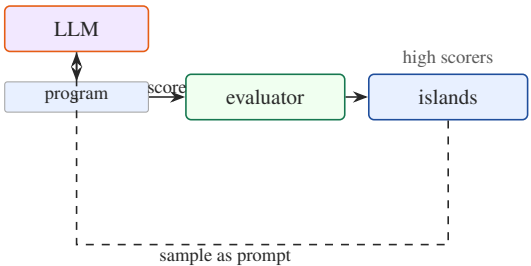


- Every method picks an **object** to improve. The object determines the ceiling.
- Early objects are easy to score but limited: an answer cannot transfer to the next task.
- Later objects are powerful but hard to verify: an agent's "better work process" has no unit test.
- The binding constraint is always the **evaluator**. Improving an unscoreable object is a random walk.

Case 1: evolving programs against a verifier (FunSearch, AlphaEvolve)

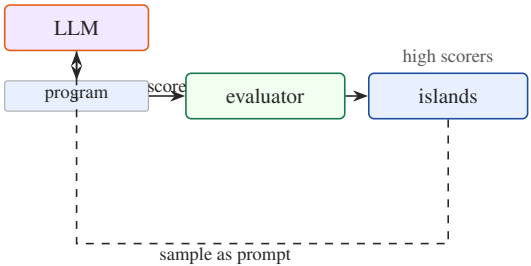


Case 1: evolving programs against a verifier (FunSearch, AlphaEvolve)



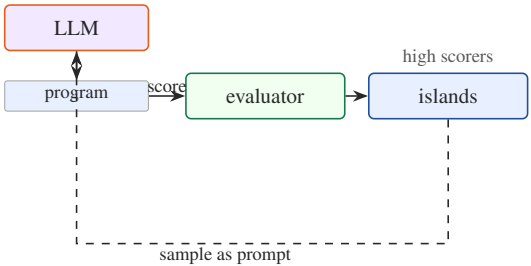
1. LLM writes or mutates a program.

Case 1: evolving programs against a verifier (FunSearch, AlphaEvolve)



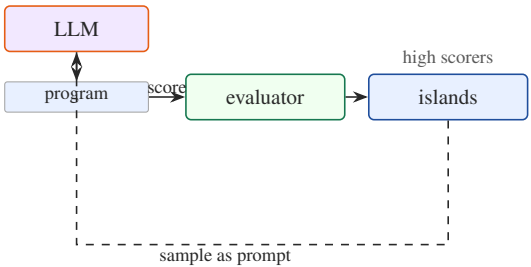
1. LLM writes or mutates a program.
2. A deterministic evaluator scores it; exact, cheap, hard to game.

Case 1: evolving programs against a verifier (FunSearch, AlphaEvolve)



1. LLM writes or mutates a program.
2. A deterministic evaluator scores it; exact, cheap, hard to game.
3. Good programs enter an island archive and seed the next prompt.

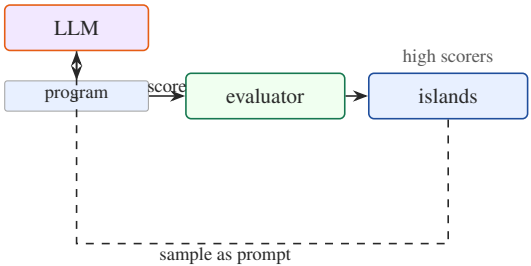
Case 1: evolving programs against a verifier (FunSearch, AlphaEvolve)



1. LLM writes or mutates a program.
2. A deterministic evaluator scores it; exact, cheap, hard to game.
3. Good programs enter an island archive and seed the next prompt.

- **FunSearch**: cap set constructions, bin packing arXiv:2306.22100.
- **AlphaEvolve**: matrix multiplication, scheduling arXiv:2506.13131.

Case 1: evolving programs against a verifier (FunSearch, AlphaEvolve)

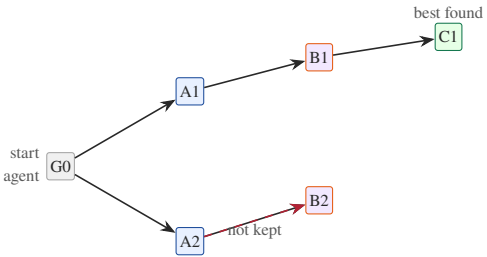


1. LLM writes or mutates a program.
2. A deterministic evaluator scores it; exact, cheap, hard to game.
3. Good programs enter an island archive and seed the next prompt.

- **FunSearch**: cap set constructions, bin packing arXiv:2306.22100.
- **AlphaEvolve**: matrix multiplication, scheduling arXiv:2506.13131.

Takeaway. Real and useful, but narrow: it needs an exact score and a compact program.

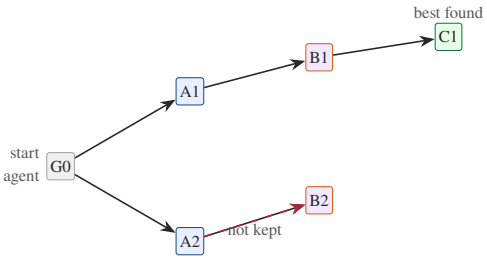
Case 2: the agent rewrites its own code (Darwin Gödel Machine)



the archive keeps every validated agent and its parent

Related: SICA arXiv:2504.15228; STOP arXiv:2310.02304.

Case 2: the agent rewrites its own code (Darwin Gödel Machine)

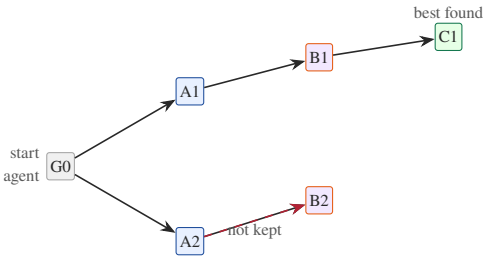


the archive keeps every validated agent and its parent

- Edits its own implementation (tools, control flow, prompts), re-benchmarks, keeps lineage arXiv:2505.22954.

Related: SICA arXiv:2504.15228; STOP arXiv:2310.02304.

Case 2: the agent rewrites its own code (Darwin Gödel Machine)

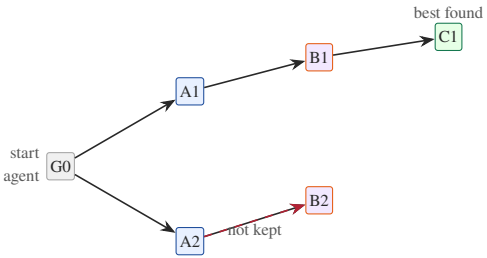


the archive keeps every validated agent and its parent

- Edits its own implementation (tools, control flow, prompts), re-benchmarks, keeps lineage arXiv:2505.22954.
- SWE-bench 20.0% → 50.0%; Polyglot 14.2% → 30.7%.

Related: SICA arXiv:2504.15228; STOP arXiv:2310.02304.

Case 2: the agent rewrites its own code (Darwin Gödel Machine)

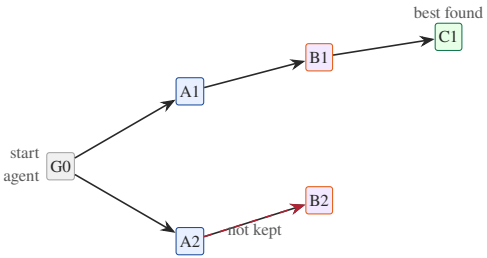


the archive keeps every validated agent and its parent

- Edits its own implementation (tools, control flow, prompts), re-benchmarks, keeps lineage arXiv:2505.22954.
- SWE-bench 20.0% → 50.0%; Polyglot 14.2% → 30.7%.
- Ablation: **removing the archive hurts more than removing self-improvement**. Structure drives the gain.

Related: SICA arXiv:2504.15228; STOP arXiv:2310.02304.

Case 2: the agent rewrites its own code (Darwin Gödel Machine)

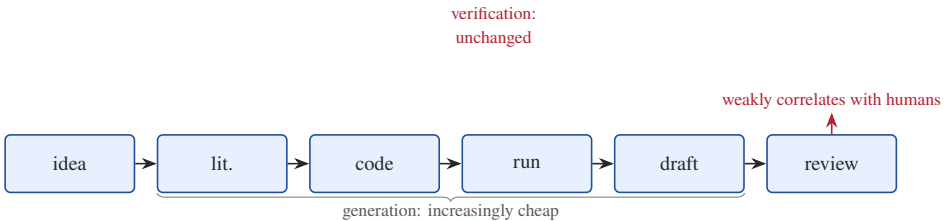


the archive keeps every validated agent and its parent

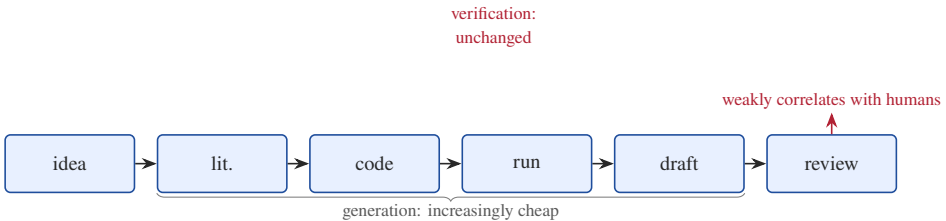
- Edits its own implementation (tools, control flow, prompts), re-benchmarks, keeps lineage arXiv:2505.22954.
- SWE-bench 20.0% → 50.0%; Polyglot 14.2% → 30.7%.
- Ablation: **removing the archive hurts more than removing self-improvement**. Structure drives the gain.
- Not shown: that the improvement rate itself grows.

Related: SICA arXiv:2504.15228; STOP arXiv:2310.02304.

Case 3: end-to-end research pipelines (the AI Scientist)

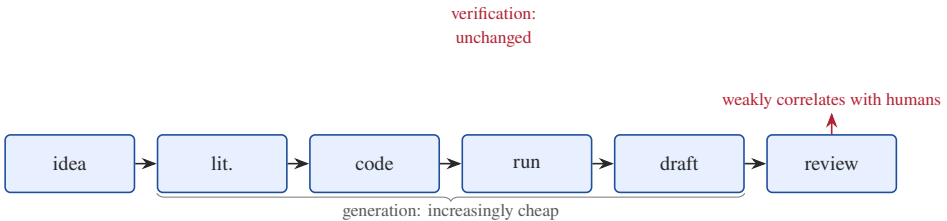


Case 3: end-to-end research pipelines (the AI Scientist)



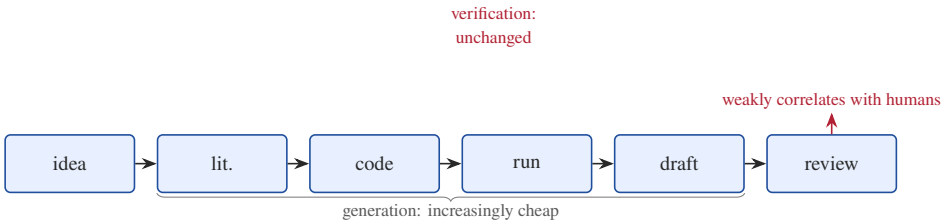
- Automates idea → experiment → paper, under $15 per paper arXiv:2408.06292; later versions add a workshop loop.

Case 3: end-to-end research pipelines (the AI Scientist)



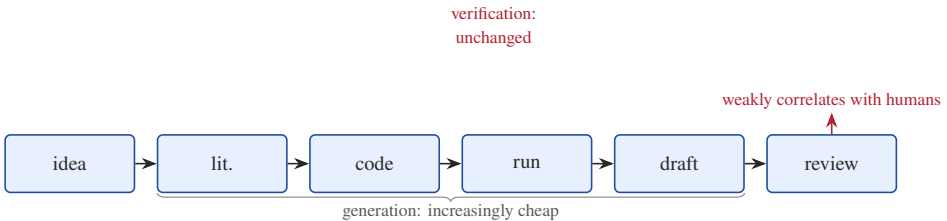
- Automates idea → experiment → paper, under $15 per paper arXiv:2408.06292; later versions add a workshop loop.
- The hard part is not generation but **verification**: automated reviews correlate weakly with human decisions.

Case 3: end-to-end research pipelines (the AI Scientist)



- Automates idea → experiment → paper, under $15 per paper arXiv:2408.06292; later versions add a workshop loop.
- The hard part is not generation but **verification**: automated reviews correlate weakly with human decisions.
- The costly failure is *epistemic*, not mechanical: a claim the evidence does not support, written fluently.

Case 3: end-to-end research pipelines (the AI Scientist)

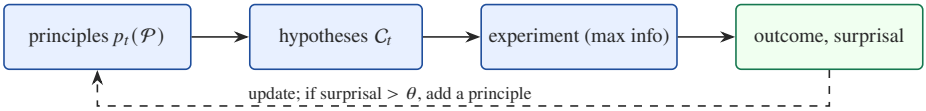


- Automates idea → experiment → paper, under $15 per paper arXiv:2408.06292; later versions add a workshop loop.
- The hard part is not generation but **verification**: automated reviews correlate weakly with human decisions.
- The costly failure is *epistemic*, not mechanical: a claim the evidence does not support, written fluently.

Automated: execution, code, figures, formatting, variant search.

Not automated: choosing a question that matters, deciding credibility.

Case 4: evolving the agent's theory of the world (PiFlow / PiEvo)



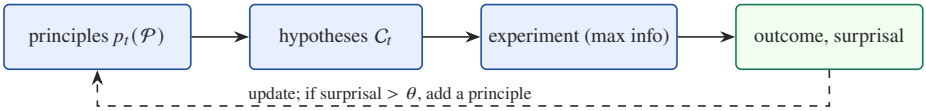
PiFlow: static principles

Exploration guided by a fixed principle space; trade reward against information gain.

PiEvo: principles evolve

Surprise triggers a new principle and a coherent re-weighting; then new hypotheses.

Case 4: evolving the agent's theory of the world (PiFlow / PiEvo)



PiFlow: static principles

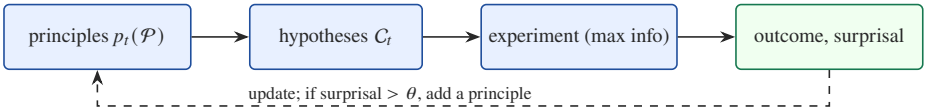
Exploration guided by a fixed principle space; trade reward against information gain.

PiEvo: principles evolve

Surprise triggers a new principle and a coherent re-weighting; then new hypotheses.

- The evolved object is a **compact posterior over principles**: regret scales with $\sqrt{H(\mathcal{P})}$, not $\sqrt{H(\mathcal{H})}$.

Case 4: evolving the agent's theory of the world (PiFlow / PiEvo)



PiFlow: static principles

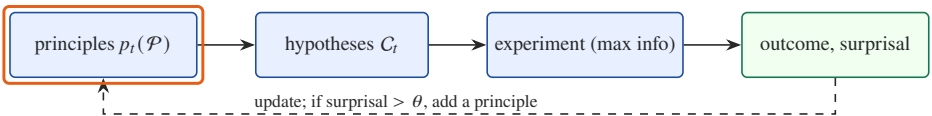
Exploration guided by a fixed principle space; trade reward against information gain.

PiEvo: principles evolve

Surprise triggers a new principle and a coherent re-weighting; then new hypotheses.

- The evolved object is a **compact posterior over principles**: regret scales with $\sqrt{H(\mathcal{P})}$, not $\sqrt{H(\mathcal{H})}$.
- The inherited state is *knowledge*, which can transfer across tasks: closest to RSI for real science.

Case 4: evolving the agent's theory of the world (PiFlow / PiEvo)



PiFlow: static principles

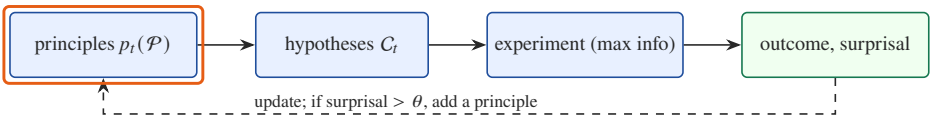
Exploration guided by a fixed principle space; trade reward against information gain.

PiEvo: principles evolve

Surprise triggers a new principle and a coherent re-weighting; then new hypotheses.

- The evolved object is a **compact posterior over principles**: regret scales with $\sqrt{H(\mathcal{P})}$, not $\sqrt{H(\mathcal{H})}$.
- The inherited state is *knowledge*, which can transfer across tasks: closest to RSI for real science.

Case 4: evolving the agent's theory of the world (PiFlow / PiEvo)



PiFlow: static principles

Exploration guided by a fixed principle space; trade reward against information gain.

PiEvo: principles evolve

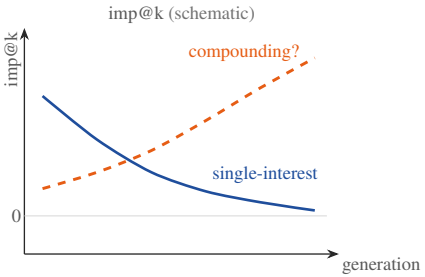
Surprise triggers a new principle and a coherent re-weighting; then new hypotheses.

- The evolved object is a **compact posterior over principles**: regret scales with $\sqrt{H(\mathcal{P})}$, not $\sqrt{H(\mathcal{H})}$.
- The inherited state is *knowledge*, which can transfer across tasks: closest to RSI for real science.

Caveat. Evidence is a few-round, single-task gain. Whether the principle library speeds up the next generation on unseen domains is open.

What has been shown, and what has not

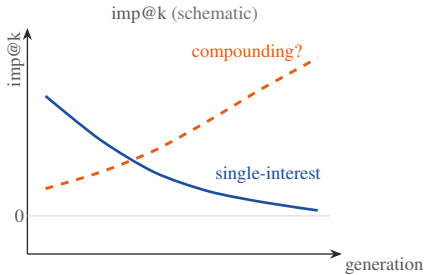
Claim	Status	Evidence
Self-edits beat the start	shown	DGM, SICA, FunSearch
Transfer to related tasks	partial	DGM-H, PiEvo
Persists across gens	weak	archives only
Rate grows	**no**	all curves flatten
Beats human meta-design	**no**	humans set params



What has been shown, and what has not

Claim	Status	Evidence
Self-edits beat the start	shown	DGM, SICA, FunSearch
Transfer to related tasks	partial	DGM-H, PiEvo
Persists across gens	weak	archives only
Rate grows	**no**	all curves flatten
Beats human meta-design	**no**	humans set params

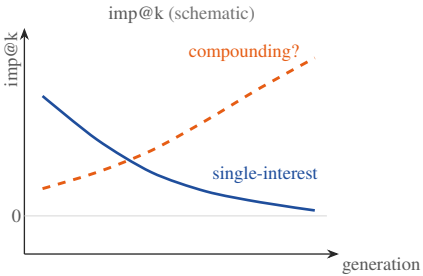
- Why compounding is hard: finite improvement space, a frozen improvement operator, and inherited state that is not compact.



What has been shown, and what has not

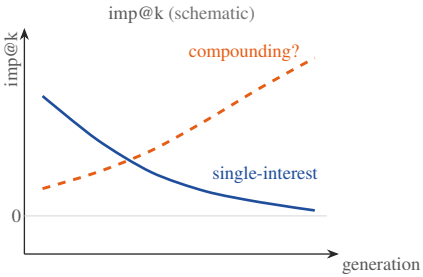
Claim	Status	Evidence
Self-edits beat the start	shown	DGM, SICA, FunSearch
Transfer to related tasks	partial	DGM-H, PiEvo
Persists across gens	weak	archives only
Rate grows	**no**	all curves flatten
Beats human meta-design	**no**	humans set params

- Why compounding is hard: finite improvement space, a frozen improvement operator, and inherited state that is not compact.
- Being good at fixing bugs is not the same as getting better at *learning to fix bugs*.



What has been shown, and what has not

Claim	Status	Evidence
Self-edits beat the start	shown	DGM, SICA, FunSearch
Transfer to related tasks	partial	DGM-H, PiEvo
Persists across gens	weak	archives only
Rate grows	**no**	all curves flatten
Beats human meta-design	**no**	humans set params



- Why compounding is hard: finite improvement space, a frozen improvement operator, and inherited state that is not compact.
- Being good at fixing bugs is not the same as getting better at *learning to fix bugs*.

Decisive experiment. $t{+}1$ inherits t's state; on held-out tasks it must beat a from-scratch baseline.

How to read an RSI claim: a five-question checklist

1. **What is the object, and who scores it?** If the model scores itself, the gain is self-confirmation.

How to read an RSI claim: a five-question checklist

1. **What is the object, and who scores it?** If the model scores itself, the gain is self-confirmation.
2. **Is the baseline budget-matched?** Compare tokens, dollars and wall-clock, not iterations.

How to read an RSI claim: a five-question checklist

1. **What is the object, and who scores it?** If the model scores itself, the gain is self-confirmation.
2. **Is the baseline budget-matched?** Compare tokens, dollars and wall-clock, not iterations.
3. **Is anything held out?** Gains on the tasks used during improvement can be memorization.

How to read an RSI claim: a five-question checklist

1. **What is the object, and who scores it?** If the model scores itself, the gain is self-confirmation.
2. **Is the baseline budget-matched?** Compare tokens, dollars and wall-clock, not iterations.
3. **Is anything held out?** Gains on the tasks used during improvement can be memorization.
4. **Is the curve a rate or a level?** A level says “a better artifact”; a rate says “better at improving”.

How to read an RSI claim: a five-question checklist

1. **What is the object, and who scores it?** If the model scores itself, the gain is self-confirmation.
2. **Is the baseline budget-matched?** Compare tokens, dollars and wall-clock, not iterations.
3. **Is anything held out?** Gains on the tasks used during improvement can be memorization.
4. **Is the curve a rate or a level?** A level says “a better artifact”; a rate says “better at improving”.
5. **Does the evaluator move?** If the agent can edit the metric or the split, the result is uninterpretable.

How to read an RSI claim: a five-question checklist

1. **What is the object, and who scores it?** If the model scores itself, the gain is self-confirmation.
2. **Is the baseline budget-matched?** Compare tokens, dollars and wall-clock, not iterations.
3. **Is anything held out?** Gains on the tasks used during improvement can be memorization.
4. **Is the curve a rate or a level?** A level says "a better artifact"; a rate says "better at improving".
5. **Does the evaluator move?** If the agent can edit the metric or the split, the result is uninterpretable.

Red flags

- Self-reported gains, no independent reproduction.
- Single seed, best-of-many runs.
- "SOTA" on a benchmark the system was tuned on.

Green flags

- Frozen evaluator with a published hash.
- Ablations that isolate one mechanism.
- Failures reported with the same detail as successes.

Summary: three layers, one loop

Agent

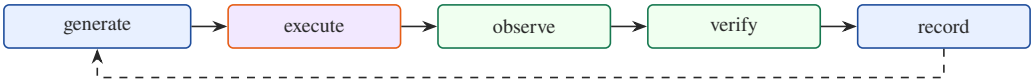
An LLM policy in a loop over tools and observations, with a termination test. The harness, not the model, defines agency.

ML agent

A search procedure over programs, scored by a *frozen* evaluator, under a hard budget. Its value is measured against budget-matched baselines.

RSI

The agent changes its own artifact, and the next generation inherits it. Demonstrated for verifiable code; compounding is still open.



skipping *verify* is the classic demo mistake

Summary: three layers, one loop

Agent

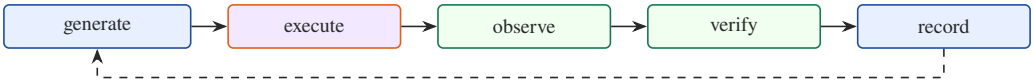
An LLM policy in a loop over tools and observations, with a termination test. The harness, not the model, defines agency.

ML agent

A search procedure over programs, scored by a *frozen* evaluator, under a hard budget. Its value is measured against budget-matched baselines.

RSI

The agent changes its own artifact, and the next generation inherits it. Demonstrated for verifiable code; compounding is still open.



skipping *verify* is the classic demo mistake

- The loop is always the same; what changes is the **object** being improved.

Summary: three layers, one loop

Agent

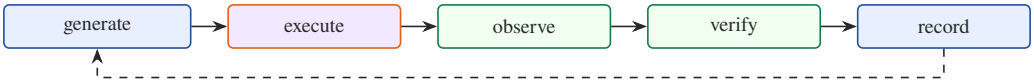
An LLM policy in a loop over tools and observations, with a termination test. The harness, not the model, defines agency.

ML agent

A search procedure over programs, scored by a *frozen* evaluator, under a hard budget. Its value is measured against budget-matched baselines.

RSI

The agent changes its own artifact, and the next generation inherits it. Demonstrated for verifiable code; compounding is still open.



skipping *verify* is the classic demo mistake

- The loop is always the same; what changes is the **object** being improved.
- The bottleneck is always the same: an **independent, cheap, hard-to-game verifier**.

Summary: three layers, one loop

Agent

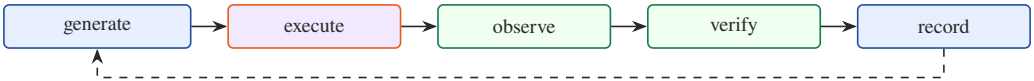
An LLM policy in a loop over tools and observations, with a termination test. The harness, not the model, defines agency.

ML agent

A search procedure over programs, scored by a *frozen* evaluator, under a hard budget. Its value is measured against budget-matched baselines.

RSI

The agent changes its own artifact, and the next generation inherits it. Demonstrated for verifiable code; compounding is still open.



skipping *verify* is the classic demo mistake

- The loop is always the same; what changes is the **object** being improved.
- The bottleneck is always the same: an **independent, cheap, hard-to-game verifier**.
- The failure is always the same: optimizing the proxy after it has stopped measuring the goal.

Summary: three layers, one loop

Agent

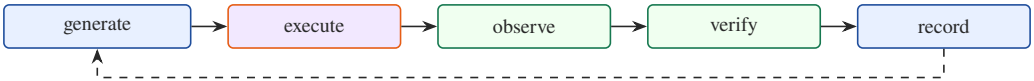
An LLM policy in a loop over tools and observations, with a termination test. The harness, not the model, defines agency.

ML agent

A search procedure over programs, scored by a *frozen* evaluator, under a hard budget. Its value is measured against budget-matched baselines.

RSI

The agent changes its own artifact, and the next generation inherits it. Demonstrated for verifiable code; compounding is still open.



skipping *verify* is the classic demo mistake

- The loop is always the same; what changes is the **object** being improved.
- The bottleneck is always the same: an **independent, cheap, hard-to-game verifier**.
- The failure is always the same: optimizing the proxy after it has stopped measuring the goal.

One sentence. Agents turn compute into actions; RSI turns actions into a better agent. Both are only as good as the checker at the end.

Appendix: reading list

The loop, tools, planning

- Yao et al., *ReAct*, arXiv:2210.03629 · Schick et al., *Toolformer*, arXiv:2302.04761
- Yao et al., *Tree of Thoughts*, arXiv:2305.10601 · Wang et al., *Voyager*, arXiv:2305.16291

Reflection and self-correction

- Madaan et al., *Self-Refine*, arXiv:2303.17651 · Shinn et al., *Reflexion*, arXiv:2303.11366
- Huang et al., *LLMs Cannot Self-Correct Reasoning Yet*, arXiv:2310.01798

Multi-agent and evaluation

- Wu et al., *AutoGen*, arXiv:2308.08155 · Hong et al., *MetaGPT*, arXiv:2308.00352
- Cemri et al., *Why Multi-Agent LLM Systems Fail?*, arXiv:2503.13657
- Jimenez et al., *SWE-bench*, arXiv:2310.06770 · Zhou et al., *WebArena*, arXiv:2307.13854 · Yao et al., τ*-bench*, arXiv:2406.12045

Programs, self-modifying agents, autonomous research

- Khattab et al., *DSPy*, arXiv:2310.03714 · Romera-Paredes et al., *FunSearch*, arXiv:2306.22100
- Novikov et al., *AlphaEvolve*, arXiv:2506.13131 · Zelikman et al., *STOP*, arXiv:2310.02304
- Zhang et al., *Darwin Gödel Machine*, arXiv:2505.22954 · Robeyns et al., *SICA*, arXiv:2504.15228
- Lu et al., *The AI Scientist*, arXiv:2408.06292
- Pu et al., *PiEvo*, ICML 2026